



Department of Physics, IIT Indore

ChaQra: A Discrete-Event Quantum Key Distribution Network Simulator

Final Project Report

AUTHOR

Prince Kumar (230051013)

SUPERVISOR

Dr. Shashank Gupta

2026-08-03

Abstract

This report documents the complete design, implementation, and simulation-based experimental evaluation of **ChaQra**, a full-stack discrete-event Quantum Key Distribution (QKD) network simulator. The project spans six weeks of progressive development, beginning with framework evaluation and concluding with a deployed, interactive web-based simulation tool.

The simulator is built on the **SeQUeNCe** quantum network simulation framework and implements two complementary runtime-selected engines: (1) a direct single-hop BBM92 simulation using a custom `TrackedSPDCSource` that correctly routes entangled photon pairs through lossy fiber `QuantumChannel` components, and (2) a multi-hop quantum repeater engine that performs entanglement generation and swapping across arbitrary user-defined network topologies using a Breadth-First Search path discovery algorithm.

The system addresses four non-trivial integration issues encountered while using the **SeQUeNCe** framework: specifically the SPDC channel bypass issue, Poisson zero-emission dropouts, `FreeQuantumState` non-subscriptability, and a 4-dimensional to 2-dimensional state-collapse matrix mismatch each of which is documented with root cause analysis and the applied fix.

Simulation-based experimental evaluation across multiple scales confirms the canonical quantum repeater advantage: over a 30 km span, a 4-hop repeater network yields a longer sifted key (135 bits vs. 110 bits) and a lower QBER (5.19% vs. 7.27%) than a direct 2-hop link. All configurations evaluated satisfy the 11% QBER security threshold for BBM92. A full-featured interactive interface featuring drag-and-drop topology editing, real-time GIS geographic mapping with automatic Haversine distance computation, per-trial simulation playback, and optical component schematics is deployed at <https://quantum-qkd-taupe.vercel.app/>.

Contents

Table of Contents	ii
1 Introduction	1
2 Week 1 — Framework Evaluation and Problem Formulation	2
2.1 Objective	2
2.2 Framework Evaluation: NetSquid vs. SeQUeNCe	2
2.3 Protocol Selection	3
3 Week 2 — Discrete Event Simulation Foundations	3
3.1 Objective	3
3.2 The DES Execution Model	3
3.3 Store Open/Close Event Loop	4
3.4 Quantum Network Component Primitives	5
4 Week 3 — Entanglement Management and the Barrett-Kok Protocol	6
4.1 Objective	6
4.2 The Fundamental Problem: Exponential Fiber Attenuation	6
4.3 The Barrett-Kok Protocol	6
4.3.1 Step 1: Spin-Photon Entanglement	7
4.3.2 Step 2: Path Erasure at the BSM Station	7
4.3.3 Step 3: The 50% Fundamental Limit	7
4.4 Entanglement Swapping: Extending the Network	8
5 Week 4 — BBM92 Protocol: Full Implementation	8
5.1 Objective	8
5.2 Protocol Description	8
5.3 SeQUeNCe Implementation	9
5.3.1 TrackedSPDCSource	9
5.3.2 SimParticipantProtocol	11
5.4 Simulation Results — Single-Hop Validation	13
6 Weeks 5 & 6 — Full-Stack Simulator: FastAPI Backend and Interactive Frontend	13
6.1 Objective	14
6.2 System Overview	14
6.3 Backend Architecture	14
6.3.1 Topology Ingestion	14
6.3.2 Engine Selection via BFS Path Discovery	15
6.3.3 Multi-Hop Quantum Repeater Engine	16

6.4	Bell State Measurement Node Protocol	17
6.5	Multi-Hop Router Node Stack	17
7	Visualization Interface	18
7.1	Dual-Mode Topology Editor	18
7.1.1	Schematic Canvas Mode	18
7.1.2	GIS Geographic Map Mode	18
7.2	Optical Component Schematics	19
7.3	Simulation Playback	19
8	Simulation-Based Experimental Evaluation	20
8.1	Experiment 1 — Short-Distance Links (100 Trials)	20
8.1.1	Channel Parameters	20
8.1.2	Results and Sifted Keys	20
8.2	Experiment 2 — Long-Distance Links (1000 Trials)	21
8.2.1	Channel Parameters	21
8.2.2	Results	21
8.3	Physical Interpretation and Cross-Experiment Analysis	22
8.3.1	Repeater Advantage at Longer Distances	22
8.3.2	QBER Behaviour Across Experiments	23
8.3.3	Security Assessment	24
9	Summary and Conclusions	24
9.1	Limitations and Future Work	25
A	Appendix A — Theoretical Foundations	27
A.1	Quantum Entanglement and Bell States	27
A.2	Measurement Bases	27
A.3	Security of BBM92	28
A.4	Spontaneous Parametric Down-Conversion	28
B	Appendix B — Full Implementation Reference	28
B.1	Repository Structure	28
B.2	Backend API	30
B.3	Simulation Request Schema	30
B.4	Simulation Response Schema	31
C	Appendix C — BSM Correction Table	32
	References	33

1 Introduction

Quantum Key Distribution (QKD) is a cryptographic technique that exploits the fundamental laws of quantum mechanics—specifically, wave-function collapse upon measurement and the no-cloning theorem—to guarantee information-theoretically secure key exchange between two parties. Unlike classical cryptographic schemes whose security rests on computational hardness assumptions, QKD provides unconditional security: any eavesdropping attempt is detectable through measurable disturbances to the quantum channel.

The BBM92 protocol [1], proposed by Bennett, Brassard, and Mermin in 1992, is the entanglement-based analogue of the prepare-and-measure BB84 protocol. Rather than one party preparing and transmitting qubits, a central Spontaneous Parametric Down-Conversion (SPDC) source distributes maximally entangled photon pairs $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|HH\rangle + |VV\rangle)$ to two communicating parties (Alice and Bob). Both parties independently measure their photons in randomly chosen polarization bases. The quantum correlations of the Bell state guarantee that whenever they happen to choose the same measurement basis, their results are perfectly correlated, forming the raw material for a shared secret key.

Simulating QKD networks under realistic physical conditions is essential both for protocol research and for engineering network deployments. Discrete-event simulation (DES) is the standard modeling paradigm for this purpose: the state of the network evolves only in response to discrete events (photon emissions, channel losses, detector clicks, classical message arrivals), allowing complex multi-node topologies to be modeled efficiently on classical hardware.

ChaQra (Quantum) is the simulation system developed over this project. It provides:

- A discrete-event simulation backend (Python/FastAPI) built on the SeQUeNCe [2] framework, supporting both direct BBM92 and multi-hop quantum repeater topologies.

- A web-based interactive frontend (React/Vite) with drag-and-drop topology design, real GIS coordinate mapping, per-trial animation, and node-level optical hardware schematics.
- End-to-end simulation-based experimental evaluation on realistic network configurations with documented physical channel parameters and measured sifted key outputs.

The remainder of this report is organized chronologically by development phase, followed by a comprehensive technical description of the final system architecture, simulation results, and appendices covering the high-level theoretical foundations and the full implementation.

2 Week 1 — Framework Evaluation and Problem Formulation

2.1 Objective

The primary objective of Week 1 was to establish a clear understanding of the project scopes—simulating the BBM92 entanglement-based QKD protocol over a multi-node quantum network and to select the most appropriate software framework for implementing the discrete-event simulation.

2.2 Framework Evaluation: NetSquid vs. SeQUeNCe

Two mature quantum network simulation frameworks were evaluated: NetSquid [3] and SeQUeNCe [2].

Table 1: Comparative evaluation of quantum network simulation frameworks.

Criterion	NetSquid	SeQUeNCe
Learning curve	Lower	Higher
Community registration	Closed (no new signups)	Open (GitHub, PyPI)
Active maintenance	Inactive	Active
Component library	Moderate	Comprehensive
Documentation	Limited tutorials	Full API reference + tutorials

Decision: SeQUeNCe was selected as the simulation platform. Although NetSquid presents a shallower learning curve, its community registration was closed at the time of evaluation, making it inaccessible. SeQUeNCe provides a fully open-source, actively maintained codebase with a modular component model, comprehensive documentation, and built-in support for quantum memories, entanglement management protocols, and SPDC photon pair sources all directly relevant to the project requirements.

2.3 Protocol Selection

Based on the project brief, the **BBM92** protocol was selected as the primary target. BBM92 requires modeling entangled photon pair generation (SPDC source), lossy quantum channels, polarization-basis measurement, classical basis sifting, and QBER estimation. A well-scoped, complete end-to-end QKD pipeline suitable for a multi-week simulation project.

3 Week 2 — Discrete Event Simulation Foundations

3.1 Objective

The objective of Week 2 was to master the SeQUeNCe DES execution kernel and prototype the elementary network building blocks required for subsequent quantum protocol simulations.

3.2 The DES Execution Model

Discrete Event Simulation models a system as a sequence of state-changing events, each occurring at a precise simulation timestamp. The SeQUeNCe DES kernel is built around three core abstractions:

- **Timeline:** Maintains a min-heap priority queue of scheduled events ordered by simulation time. `tl.run()` dequeues and executes events in chronological order until the queue is empty

or a stop-time is reached.

- **Event(*t*, *process*)**: Represents a state-change action to be fired at simulation time *t*.
- **Process(*obj*, '*method*', *args*)**: Encapsulates a deferred method call on a simulation object, executed when the event fires.

This abstraction allows complex time-ordered interaction chains to be specified declaratively without manual time-tracking logic.

3.3 Store Open/Close Event Loop

A practical demonstration was implemented as a retail store that opens and closes on a fixed 12-unit cycle. This exercise introduced cascading event scheduling each event schedules its successor and the SeQUeNCe logging framework:

```
1 class Store:
2     def __init__(self, tl: Timeline):
3         self.opening = False
4         self.timeline = tl
5
6     def open(self) -> None:
7         if self.timeline.now() >= 60:
8             self.timeline.stop()
9             log.logger.info('Store being opened.')
10            self.opening = True
11            process = Process(self, 'close', [])
12            event = Event(self.timeline.now() + 12, process)
13            self.timeline.schedule(event)
14
15    def close(self) -> None:
```



```
16     if self.timeline.now() >= 60:
17         self.timeline.stop()
18         log.logger.info('Store being closed.')
19         self.opening = False
20         process = Process(self, 'open', [])
21         event = Event(self.timeline.now() + 12, process)
22         self.timeline.schedule(event)
```

Listing 1: Self-scheduling DES event loop (DES/des.py)

3.4 Quantum Network Component Primitives

Beyond the DES kernel, the SeQUeNCe framework provides pre-built quantum hardware models. The following components were studied and prototyped this week:

- **Memory:** Quantum memory with configurable fidelity, coherence_time (s), efficiency, frequency (Hz), and wavelength (nm).
- **QSDetectorPolarization:** Hardware single-photon polarization detector with configurable detection efficiency and dark count rate.
- **QuantumChannel:** Fiber channel applying Poisson-distributed photon loss (parameterized by attenuation in dB/m) and random polarization state flipping (parameterized by polarization_fidelity).
- **ClassicalChannel:** Ideal classical message channel with configurable propagation delay.
- **Protocol:** Abstract base class for node-local state machines that exchange typed Message objects over classical channels.

A bidirectional ping-pong protocol (hardware-module/pingpong.py) was implemented to

validate classical channel communication between two nodes, confirming the message-passing architecture that would later be used for basis sifting in BBM92.

4 Week 3 — Entanglement Management and the Barrett-Kok Protocol

4.1 Objective

The objective of Week 3 was to understand the theoretical barrier to long-distance quantum communication and to implement the entanglement generation mechanism at the heart of quantum repeater networks.

4.2 The Fundamental Problem: Exponential Fiber Attenuation

Photon transmission probability over a fiber of length L (m) with attenuation coefficient α (dB/m) is:

$$P_{\text{trans}}(L) = 10^{-\alpha L/10} \quad (1)$$

For standard telecom fiber ($\alpha \approx 0.0002$ dB/m at 1550 nm), over $L = 100$ km, $P_{\text{trans}} \approx 10^{-2}$ a 99% loss rate. Classical repeaters amplify signals by measuring and retransmitting, but this is forbidden for quantum states by the **No-Cloning Theorem**: a general unknown quantum state $|\psi\rangle$ cannot be copied. Measuring for amplification collapses the superposition irreversibly.

4.3 The Barrett-Kok Protocol

The Barrett-Kok protocol provides a solution: instead of transmitting quantum states across long distances, entanglement is generated between adjacent short-distance nodes using heralded photon pair measurement at a central Bell State Measurement (BSM) station. The protocol proceeds in three steps.

4.3.1 Step 1: Spin-Photon Entanglement

Each node initializes its quantum memory in superposition $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. A laser pulse causes conditional photon emission from $|1\rangle$ only:

$$\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \xrightarrow{\text{laser}} \frac{1}{\sqrt{2}}(|0\rangle|\text{vac}\rangle + |1\rangle|\text{photon}\rangle) \quad (2)$$

The emitted photon carries information about the qubit state they are now entangled.

4.3.2 Step 2: Path Erasure at the BSM Station

Photons from nodes A and B travel to a central 50:50 beam splitter. A detector click on either output port projects the joint state. The key insight is that a single click does *not* identify which node emitted the photon path information is erased. This enforces the memory-photon entanglement to collapse the two remote memories into the Bell state:

$$|\Psi^+\rangle_{A,B} = \frac{1}{\sqrt{2}}(|0\rangle_A|1\rangle_B + |1\rangle_A|0\rangle_B) \quad (3)$$

4.3.3 Step 3: The 50% Fundamental Limit

Linear-optics BSMs using beam splitters and photo-detectors can distinguish only two of the four Bell states the antisymmetric states $|\Psi^+\rangle$ and $|\Psi^-\rangle$ (via Hong-Ou-Mandel anti-bunching) while the symmetric states $|\Phi^\pm\rangle$ produce identical detector signatures (two clicks, same port). Since all four Bell states occur with equal probability of 25%, the maximum success probability for any linear-optics BSM is:

$$P_{\text{BK success}} \leq 50\% \quad (4)$$

Simulation across 1000 independent protocol attempts confirmed this bound: approximately 500 successes and 500 failures.

4.4 Entanglement Swapping: Extending the Network

Once short-distance entangled links (A, R_1) and (R_2, B) are established at an intermediate repeater R , the entanglement range can be extended to the full (A, B) span without any direct photon exchange:

$$\begin{aligned}
 |\Psi\rangle_{\text{total}} &= |\Phi^+\rangle_{A,R_1} \otimes |\Phi^+\rangle_{R_2,B} \\
 &= \frac{1}{2} \left[|\Phi^+\rangle_{R_1,R_2} |\Phi^+\rangle_{A,B} + |\Phi^-\rangle_{R_1,R_2} |\Phi^-\rangle_{A,B} \right. \\
 &\quad \left. + |\Psi^+\rangle_{R_1,R_2} |\Psi^+\rangle_{A,B} + |\Psi^-\rangle_{R_1,R_2} |\Psi^-\rangle_{A,B} \right]
 \end{aligned} \tag{5}$$

The repeater R performs a local BSM on R_1 and R_2 , projecting the A - B pair into the corresponding Bell state. The BSM outcome (2 classical bits) is sent to B over a classical channel, which applies the appropriate Pauli correction to recover the target state $|\Phi^+\rangle_{A,B}$.

5 Week 4 — BBM92 Protocol: Full Implementation

5.1 Objective

The objective of Week 4 was to implement a complete, end-to-end BBM92 QKD protocol pipeline and validate it against theoretical predictions under realistic channel parameters.

5.2 Protocol Description

The BBM92 protocol [1] distributes quantum key bits by exploiting the correlations of polarization-entangled photon pairs. The five protocol stages are:

1. **Entangled pair distribution:** A central SPDC source emits photon pairs in the $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|HH\rangle + |VV\rangle)$ Bell state. One photon is directed to Alice, the other to Bob, via separate quantum fiber channels.
2. **Independent basis selection:** Alice and Bob each independently and uniformly at random choose a measurement basis: Z -basis $\{|H\rangle, |V\rangle\}$ (basis code 0) or X -basis $\{|+\rangle, |-\rangle\}$ (basis code 1). The key property of $|\Phi^+\rangle$ is that same-basis measurements always produce identical outcomes, while different-basis measurements produce uncorrelated results.
3. **Polarization measurement:** Each party measures its photon and records the binary outcome.
4. **Classical basis sifting:** After all photons have been measured, Alice and Bob publicly broadcast their basis choices over an authenticated classical channel. Only trials where both chose the same basis are retained; the rest are discarded. This sifting retains $\approx 50\%$ of trials in the ideal case.
5. **QBER estimation:** A random sample ($\approx 15\%$) of the sifted key is publicly compared. The fraction of disagreements is the Quantum Bit Error Rate:

$$\text{QBER} = \frac{\sum_{i \in \text{sample}} b_{A,i} \oplus b_{B,i}}{|\text{sample}|} \times 100\% \quad (6)$$

If $\text{QBER} \leq 11\%$, the channel is classified as secure and the protocol proceeds to error correction and privacy amplification.

5.3 SeQUeNCe Implementation

The simulation was implemented using three custom classes extending the SeQUeNCe library, plus a classical post-processing sifting stage.

5.3.1 TrackedSPDCSource

SeQUeNCe's built-in SPDCSource emits photon pairs with Poisson-distributed multiplicity at rate f Hz. Each emission invokes an internal `send_photons()` method. Two critical integration issues were identified during development:

Bug 1 — Channel bypass : `SPDCSource.send_photons()` calls `Process(receiver, "get", [photon])` directly on registered receiver objects, bypassing the `QuantumChannel` instance. No attenuation loss or polarization noise is applied. This caused QBER to always read 0%.

Fix: Subclassed as `TrackedSPDCSource` and overrode `send_photons()` to call `owner.send_qubit(dst, photon)`, which internally invokes `QuantumChannel.transmit()`, correctly applying fiber loss and polarization flipping before scheduling photon arrival.

Bug 2 — Poisson zero-emission dropouts : With the default `mean_photon_num = 1.0`, the Poisson distribution gives $P(0 \text{ photons}) = e^{-1} \approx 37\%$, silently voiding 37% of trials.

Fix: Set `mean_photon_num = 10.0`, reducing the empty-trial probability to $e^{-10} \approx 5 \times 10^{-3}\%$.

Additionally, each emitted photon was stamped with a unique trial index (`photon.trial = trial_index`) so that measurement results from Alice and Bob could be accurately matched in a global `SIMULATION_TRIALS` dictionary.

```

1 class TrackedSPDCSource(SPDCSource):
2     def __init__(self, *args, **kwargs):
3         super().__init__(*args, **kwargs)
4         self._trial_index = 0
5         self.channels = (None, None)

```

```

6
7  def send_photons(self, time, photons):
8      trial = self._trial_index
9      self._trial_index += 1
10     for ph in photons:
11         ph.trial = trial
12         ph.pair_id = trial
13     ph_alice, ph_bob = photons
14     qch_a, qch_b = self.channels
15     if qch_a and qch_b:
16         self.timeline.schedule(Event(int(round(time)),
17                                     Process(self.owner, 'send_qubit', [qch_a.receiver, ph_alice])))
18         self.timeline.schedule(Event(int(round(time)),
19                                     Process(self.owner, 'send_qubit', [qch_b.receiver, ph_bob])))

```

Listing 2: TrackedSPDCSource: trial tagging and channel routing (backend/main.py)

5.3.2 SimParticipantProtocol

Each endpoint node runs a `SimParticipantProtocol` that intercepts arriving photons via a `PhotonTap` entity, randomly selects a measurement basis, and records the outcome:

```

1  def received_message(self, src: str, msg: Message):
2      trial = msg.payload.get('trial')
3      photon = msg.payload.get('photon')
4      if trial is None:
5          return
6      basis = self.rng.randrange(2)          # 0 = Z-basis, 1 = X-basis
7      info = SIMULATION_TRIALS.setdefault(trial, {})
8      info[f'{self.node_id}_basis'] = basis
9

```

```

10 # Physical hardware detector (collapses 4D joint state to 2D)
11 try:
12     self.qsdet.get(photon, src=src)
13 except Exception:
14     pass
15
16 # Logical measurement (operates on now-collapsed 2D state)
17 basis_mat = polarization['bases'][basis]
18 try:
19     res = Photon.measure(basis_mat, photon, self.rng)
20 except Exception:
21     res = None          # fallback: no detection
22
23 if res is not None:
24     info[f'{self.node_id}_result'] = res

```

Listing 3: Endpoint measurement protocol

Two additional integration issues were identified during development of `SimParticipantProtocol` with the full backend:

Bug 3 — FreeQuantumState non-subscriptable : The initial implementation directly indexed `photon.quantum_state[1]`. `SeQUeNCe`'s `FreeQuantumState` object does not support Python subscript notation, causing a `TypeError` on every measurement attempt and recording all trials as lost (zero sifted bits).

Fix: Replaced manual vector indexing with the standard `Photon.measure(basis_matrix, photon, rng)` library call.

Bug 4 — 4D→2D dimension mismatch : Two entangled photons share a 4-dimensional joint state

vector in SeQUeNCe’s state representation. Calling `Photon.measure` with a 2×2 basis matrix on an uncollapsed joint state raised `matmul: size 4 is different from 2`.

Fix: `qsdet.get(photon)` is called *first*: the `QSDetectorPolarization` model internally projects the joint 4D state onto a 2D subspace for the local photon. `Photon.measure` is then called on the already-collapsed, 2-dimensional state.

5.4 Simulation Results — Single-Hop Validation

Validation parameters: 200 trials, 1 km fiber per link, attenuation 0.0002 dB/m, polarization fidelity 0.93, SPDC frequency 100 Hz.

- **Sifted key length:** 80–110 bits ($\approx 45\%$ yield, theoretical max 50%)
- **Agreement rate:** 93–99%
- **Observed QBER:** 0–8% (expected $\approx 7\%$ from $1 - p_{\text{fidelity}} = 0.07$)

The 5% gap between observed and theoretical yield is attributed to fiber attenuation loss (2 dB over 1 km $\Rightarrow P_{\text{trans}} \approx 63\%$ per photon, so some trials lose one photon and cannot be sifted). The QBER range arises from statistical sampling variance over small (≈ 12 bit) QBER estimation samples.

A representative output:

```

1 [bob] Sifted length: 92
2 Total trials: 200
3 Sifted key length: 92
4 Agreement: 88/92 (95.65%)
5 Estimated QBER: 4.35%
6 Alice key: 0101111110010011101000011100000110100010010...
7 Bob key:   010111111001001110100001110000110100010010...
```

Listing 4: Sample output: 200-trial BBM92 run

6 Weeks 5 & 6 — Full-Stack Simulator: FastAPI Backend and Interactive Frontend

6.1 Objective

The objective of Weeks 5 and 6 was to generalize the static three-node BBM92 script into a complete, web-accessible simulation system supporting arbitrary user-defined topologies including multi-hop quantum repeater configurations with an interactive visual interface.

6.2 System Overview

The final system comprises two layers:

1. **Backend** (backend/main.py): A FastAPI server that accepts a topology description as a JSON POST request to `/api/simulate`, dynamically selects and executes the appropriate simulation engine, and returns per-trial telemetry including sifted keys and QBER.
2. **Frontend** (frontend/src/App.jsx): A React/Vite single-page application providing drag-and-drop network topology design, GIS geographic coordinate mapping, SVG optical hardware schematics, and real-time simulation playback.

The deployed instance is available at <https://quantum-qkd-taupe.vercel.app/>.

6.3 Backend Architecture

6.3.1 Topology Ingestion

Pydantic models define and validate the incoming configuration payload:

```
1 class ComponentConfig(BaseModel):  
2     id: str; name: str; type: str
```

```
3 efficiency: Optional[float] = 0.95
4 dark_count: Optional[float] = 1e-6
5 mean_photon_num: Optional[float] = 10.0
6 frequency: Optional[int] = 100
7
8 class NodeConfig(BaseModel):
9     id: str; name: str; type: str
10    components: List[ComponentConfig]
11
12 class ChannelConfig(BaseModel):
13     id: str; name: str; type: str
14     src: str; dst: str
15     distance: Optional[float] = 1000.0
16     attenuation: Optional[float] = 0.0002
17     fidelity: Optional[float] = 0.95
18     delay: Optional[float] = 1e-6
```

Listing 5: Pydantic configuration models

6.3.2 Engine Selection via BFS Path Discovery

The backend builds an adjacency graph from the quantum channel list and performs a BFS from each source node to Alice and Bob. The path length determines which engine is invoked:

```
1 def _find_path_bfs(start_id, end_id, channels):
2     adj = {}
3     for ch in channels:
4         if ch.type == 'quantum':
5             adj.setdefault(ch.src, []).append(ch.dst)
6             adj.setdefault(ch.dst, []).append(ch.src)
7     visited, queue = {start_id}, [(start_id, [start_id])]
```

```
8 while queue:
9     curr, path = queue.pop(0)
10    if curr == end_id:
11        return path
12    for nxt in adj.get(curr, []):
13        if nxt not in visited:
14            visited.add(nxt); queue.append((nxt, path + [nxt]))
15    return None
```

Listing 6: BFS path discovery on quantum channel graph

If the discovered path has ≤ 2 hops (direct link), the SeQUeNCe DES engine is used. If > 2 hops (multi-hop repeater), the physics-based Monte Carlo engine is invoked.

6.3.3 Multi-Hop Quantum Repeater Engine

The multi-hop engine simulates each hop independently using the Beer-Lambert attenuation model (Equation 1). At intermediate repeater nodes, an additional 84% memory coupling efficiency loss and a 2% fidelity degradation per operation are applied. Entanglement swapping correlates the end-to-end Alice-Bob measurement outcomes:

```
1 def simulate_path(path):
2     survived, fidelity = True, 1.0
3     for i in range(len(path) - 1):
4         ch = ch_map.get(f"{path[i]}-{path[i+1]}")
5         att_db = ch.attenuation * ch.distance
6         trans = 10 ** (-att_db / 10.0)
7         if rng.random() > trans: return False, 0.0
8         fidelity *= ch.fidelity
9         if 0 < i < len(path) - 2:           # intermediate repeater hop
10             if rng.random() > 0.84: return False, 0.0
```

```

11         fidelity *= 0.98
12     return survived, fidelity

```

Listing 7: Per-trial multi-hop chain simulation

6.4 Bell State Measurement Node Protocol

For direct topologies containing an intermediate BSM node, the `SimBSMProtocol` collects photons from two independent SPDC sources on the same trial index. When both arrive, a BSM outcome is drawn:

```

1 class SimBSMProtocol(Protocol):
2     def received_message(self, src, msg):
3         trial = msg.payload.get('trial')
4         info = SIMULATION_TRIALS.setdefault(trial, {})
5         count = info.get('bsm_photon_count', 0)
6         info['bsm_photon_count'] = count + 1
7         if info['bsm_photon_count'] == 2:
8             info['bsm_result'] = self.rng.randrange(4)
9             # 0 = |Phi+>, 1 = |Phi->, 2 = |Psi+>, 3 = |Psi->

```

Listing 8: SimBSMProtocol: joint Bell measurement

During post-processing, outcomes 1 ($|\Phi^-\rangle$) and 3 ($|\Psi^-\rangle$) trigger a classical bit correction on Bob's result: $b_B \leftarrow 1 - b_B$, restoring key correlation as derived from Equation 5.

6.5 Multi-Hop Router Node Stack

For SeQUeNCe-based multi-hop simulations, each intermediate node is instantiated as a `QKDRouterNode` custom `Node` subclass equipped with a `MemoryArray` and a rule-based resource management stack:

- **ResourceManager:** Coordinates all entanglement-related protocols running at the node.

- **MemoryManager:** Tracks the state (RAW, ENTANGLED) and remote peer of each memory slot.
- **RuleManager:** Evaluates installed rules against current memory states and triggers protocol actions.
- **EntanglementGenerationA:** Drives heralded photon emission to the adjacent BSM node; updates memory state to ENTANGLED on a BSM success notification.
- **EntanglementSwappingA:** Fired by the rule manager when a node holds two memories entangled with its left and right neighbors. Performs a local BSM, sends the correction to the right node, and frees both memories.
- **EntanglementSwappingB:** Runs at the right end-node; applies the Pauli correction received from the swapping node to finalize the end-to-end entanglement.

7 Visualization Interface

7.1 Dual-Mode Topology Editor

The frontend provides two complementary topology design modes:

7.1.1 Schematic Canvas Mode

An interactive drag-and-drop canvas for designing abstract network topologies. Nodes are typed (SPDC Source, Endpoint Receiver, BSM Node, Quantum Router) and connected by directed channel edges distinguished by type: quantum channels rendered in blue and classical channels in orange. Nodes support configurable internal hardware component parameters via a sidebar inspector.

7.1.2 GIS Geographic Map Mode

A Leaflet-based geographic map allows node placement at real-world coordinates. Channel distances are automatically computed from node latitude-longitude pairs using the Haversine formula:

$$d = 2R \arcsin\left(\sqrt{\sin^2 \frac{\Delta\phi}{2} + \cos \phi_1 \cos \phi_2 \sin^2 \frac{\Delta\lambda}{2}}\right), \quad R = 6371 \text{ km} \quad (7)$$

Propagation delay is derived as:

$$t_{\text{delay}} = \frac{d}{v_{\text{fiber}}}, \quad v_{\text{fiber}} \approx 2 \times 10^8 \text{ m/s} \quad (8)$$

Custom SVG markers differentiate node types on the map. Selecting a placed node automatically updates all connected channel parameters.

7.2 Optical Component Schematics

Each node type has an associated SVG schematic rendered in the inspector panel:

- **SPDC Source:** BBO crystal, ultraviolet pump laser input, two diverging entangled photon output beams (parametric down-conversion visualization).
- **Endpoint Receiver:** Fiber input, beam-splitting tap, Polarizing Beam Splitter (PBS), and two avalanche single-photon detector modules labeled DET-H and DET-V for horizontal and vertical polarization channels.

7.3 Simulation Playback

During and after simulation execution:

- CSS keyframe animations render discrete photon-pulse markers traveling along channel edges at a speed proportional to the channel propagation delay.

- A per-trial inspector panel shows basis choices, photon survival flags, BSM outcomes, sifting decisions, and cumulative key bits.
- Summary statistics (total trials, sifted key length, agreement rate, QBER, security status) are displayed in a telemetry panel.

8 Simulation-Based Experimental Evaluation

Two independent simulation experiments were conducted. Experiment 1 uses shorter inter-node distances (~ 4 km per hop), while Experiment 2 scales to longer links (~ 8 – 16 km) and a ten-fold larger trial count (1000 trials), allowing a clearer statistical comparison of the direct versus repeater configurations. Both experiments compare a **direct 2-hop link** (source E connected directly to end nodes A_2 and B_2) against a **4-hop quantum repeater chain** (source E via intermediate nodes A_1 , B_1 to A_2 , B_2). All quantum channels share the same attenuation coefficient of $\alpha = 0.0002$ dB/m.

8.1 Experiment 1 — Short-Distance Links (100 Trials)

8.1.1 Channel Parameters

8.1.2 Results and Sifted Keys

Sifted keys — Direct Link (24 bits):

A_2 : 001110011011000101000000

B_2 : 001110011001000101000000

Sifted keys — 4-Hop Repeater (29 bits):

A_2 : 00100111111010111100100101000

B_2 : 00100111111010111100101111000

Table 2: Experiment 1: quantum and classical channel parameters.

Channel	Direction	Distance (m)	Fidelity	Delay (s)
<i>Direct topology (2 hops)</i>				
$E \rightarrow A_2$	Quantum	4 290	0.97	2.145×10^{-5}
$E \rightarrow B_2$	Quantum	3 686	0.95	1.843×10^{-5}
$A_2 \rightarrow B_2$	Classical (sifting)	7 540	—	3.000×10^{-5}
<i>Repeater topology (4 hops)</i>				
$E \rightarrow A_1$	Quantum	4 290	0.97	2.145×10^{-5}
$E \rightarrow B_1$	Quantum	3 686	0.95	1.843×10^{-5}
$A_1 \rightarrow A_2$	Quantum	3 880	0.96	1.940×10^{-5}
$B_1 \rightarrow B_2$	Quantum	4 016	0.97	2.008×10^{-5}
$A_2 \rightarrow B_2$	Classical (sifting)	7 540	—	3.000×10^{-5}

Table 3: Experiment 1 results: direct vs. repeater topology (100 trials).

Metric	Direct Link (2 hops)	Quantum Repeater (4 hops)
Total trials	100	100
Sifted key length	24 bits	29 bits
QBER	4.17%	6.90%
Security status	SECURE	SECURE

8.2 Experiment 2 — Long-Distance Links (1000 Trials)

8.2.1 Channel Parameters

8.2.2 Results

Sifted keys — Direct Link (110 bits):

A2: 10010011001001111101111101111000001010101111001101011101011010111101100
010100000000111101101100000110000010000
B2: 10011011001001111100111101111000011010101111001101011111011010111101110
010100000000111101001100000111000010001

Sifted keys — 4-Hop Repeater (135 bits):

Table 4: Experiment 2: quantum and classical channel parameters.

Channel	Direction	Distance (m)	Fidelity	Delay (s)
<i>Direct topology (2 hops)</i>				
$E \rightarrow A_2$	Quantum	14 833	0.98	7.417×10^{-5}
$E \rightarrow B_2$	Quantum	15 745	0.98	7.873×10^{-5}
$A_2 \rightarrow B_2$	Classical (sifting)	14 024	—	7.012×10^{-5}
<i>Repeater topology (4 hops)</i>				
$E \rightarrow A_1$	Quantum	7 249	0.98	3.625×10^{-5}
$A_1 \rightarrow A_2$	Quantum	7 584	0.98	3.792×10^{-5}
$E \rightarrow B_1$	Quantum	8 270	0.98	4.135×10^{-5}
$B_1 \rightarrow B_2$	Quantum	7 490	0.98	3.745×10^{-5}
$A_2 \rightarrow B_2$	Classical (sifting)	14 024	—	7.012×10^{-5}
<i>Attenuation: 0.0002 dB/m on all quantum channels.</i>				

Table 5: Experiment 2 results: direct vs. repeater topology (1000 trials).

Metric	Direct Link (2 hops)	Quantum Repeater (4 hops)
Total trials	1 000	1 000
Sifted key length	110 bits	135 bits
Sifted key yield	11.0%	13.5%
QBER	7.27%	5.19%
Security status	SECURE	SECURE

A2: 110100111111010010001001000000000011011010000110111111001000111101101001
101101100110000101000010110010100100110001110101010000101111111
B2: 110100111101010011001001000000000011011010101110111111001000111100101001
101111100110100101000010110010100100110001110101010000101111111

8.3 Physical Interpretation and Cross-Experiment Analysis

8.3.1 Repeater Advantage at Longer Distances

The most significant result emerges in Experiment 2: at link distances of approximately 15 km per direct hop, the 4-hop repeater configuration yields *more* sifted key bits (135 vs. 110) *and* a lower

QBER (5.19% vs. 7.27%). This is the canonical demonstration of the quantum repeater advantage and was not fully resolved in Experiment 1 due to the shorter link distances.

The underlying physics is straightforward. Per-photon survival probability on the direct 15 km link is:

$$P_{\text{direct}} = 10^{-\alpha L/10} = 10^{-0.0002 \times 15,000/10} \approx 0.501 \quad (9)$$

Both photons (to A_2 and B_2) must survive for a trial to be siftable, so the joint probability is $P_{\text{direct}}^2 \approx 0.251$. In the repeater configuration, each of the four links is approximately 7.5 km, giving:

$$P_{\text{hop}} = 10^{-0.0002 \times 7,500/10} \approx 0.708 \quad (10)$$

The joint probability of two photons (each traversing one repeater hop) both surviving is $P_{\text{hop}}^2 \approx 0.501$. Even after accounting for the BSM swapping success probability (at most 50% for linear optics), the effective yield can exceed the direct-link joint survival probability when the direct link is long enough. This crossover has been experimentally confirmed here.

8.3.2 QBER Behaviour Across Experiments

Table 6 summarises the QBER measurements across both experiments and configurations.

Table 6: QBER summary across all four experimental configurations.

Experiment	Configuration	Total hop distance (m)	QBER
Exp. 1	Direct (2 hops)	$4,290 + 3,686 = 7,976$	4.17%
Exp. 1	Repeater (4 hops)	$3,880 + 4,016 = 7,896^*$	6.90%
Exp. 2	Direct (2 hops)	$14,833 + 15,745 = 30,578$	7.27%
Exp. 2	Repeater (4 hops)	$7,249 + 7,584 + 8,270 + 7,490 = 30,593$	5.19%

* Repeater-side quantum hop distances only (excluding source-to-relay hops).

In Experiment 1, the repeater QBER (6.90%) exceeds the direct QBER (4.17%) because the cumulative fidelity product across four mismatched-fidelity links ($0.97 \times 0.95 \times 0.96 \times 0.97 \approx 0.859$)

is lower than for the two direct links, and entanglement swapping adds a non-zero BSM projection noise.

In Experiment 2, all quantum links share an identical fidelity of 0.98, making the cumulative product for the four-hop chain $0.98^4 \approx 0.922$, while the direct topology's two-link product is $0.98^2 \approx 0.960$. The QBER difference is therefore smaller and the repeater's shorter individual hops provide better attenuation-corrected per-photon survival, more than compensating for the higher BSM noise, resulting in the observed inversion: lower QBER with the repeater.

8.3.3 Security Assessment

Both experiments satisfy the BBM92 security condition $\text{QBER} \leq 11\%$ for all four configurations. No adversarial intervention is detectable in any run. The BBM92 threshold is reached when photon loss forces a random-guess fallback that introduces $\approx 25\%$ errors per lost photon pair; the observed QBER values well below 11% confirm that the channels are operating in the physically realistic low-noise regime rather than the eavesdropping-induced high-error regime.

9 Summary and Conclusions

This project developed ChaQra, a full-stack discrete-event quantum network simulator implementing the BBM92 entanglement-based QKD protocol over both single-hop and multi-hop quantum repeater topologies.

The six-week development trajectory covered:

1. Systematic evaluation of quantum network simulation frameworks, leading to the adoption of SeQUeNCe.
2. Thorough internalization of the Discrete Event Simulation kernel and quantum component primitives.

3. Theoretical and empirical verification of the Barrett-Kok entanglement generation protocol and its inherent 50% linear-optics success bound.
4. Full end-to-end implementation of the BBM92 QKD protocol, resolving four non-trivial integration issues in the SeQUeNCe framework usage.
5. Generalization to a dynamic full-stack system supporting arbitrary topologies with BSM repeater nodes and geographic GIS mapping.
6. Simulation-based experimental evaluation on a realistic 4-hop fiber network topology, confirming physically consistent QBER and sifted key yields in both direct and repeater configurations.

The key technical contributions are:

- **TrackedSPDCSource:** A SeQUeNCe subclass addressing two fundamental integration issues (channel bypass and Poisson zero-emission) to produce physically consistent QKD simulations.
- **Dual simulation engine:** Runtime-selected BFS-directed dispatch between a SeQUeNCe DES engine (direct links) and a Monte Carlo physics engine (multi-hop repeaters).
- **Rule-based quantum repeater stack:** Full integration of SeQUeNCe’s EntanglementGenerationA, EntanglementSwappingA/B, and associated resource managers for automated multi-hop entanglement distribution.
- **Interactive GIS interface:** Geographic topology design with Haversine-derived channel parameters and per-trial simulation playback.

9.1 Limitations and Future Work

The current implementation has the following limitations that identify directions for future work:

- **Error correction and privacy amplification:** The simulator currently outputs raw sifted keys. Implementation of Cascade error correction and two-universal hash privacy amplification is a natural next step.
- **Eavesdropping simulation:** No active Eve node is currently modeled. Adding intercept-resend and beam-splitting attack models would allow quantitative security analysis.
- **Decoherence modeling:** Quantum memory coherence time (T_2) limits are not currently enforced as a time-out condition on entanglement swapping chains. Long-latency paths may violate memory lifetime constraints in practice.
- **Quantum error correction:** Entanglement purification protocols (BBPSSW [7]) could be integrated to improve fidelity on noisy links before key distillation.

A Appendix A — Theoretical Foundations

A.1 Quantum Entanglement and Bell States

A quantum system comprising two qubits (two-level quantum systems) spans a four-dimensional Hilbert space $\mathcal{H}_A \otimes \mathcal{H}_B$. The four maximally entangled Bell states form an orthonormal basis of this space:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|HH\rangle + |VV\rangle) \quad (11)$$

$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|HH\rangle - |VV\rangle) \quad (12)$$

$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|HV\rangle + |VH\rangle) \quad (13)$$

$$|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|HV\rangle - |VH\rangle) \quad (14)$$

In the polarization encoding used by BBM92, $|H\rangle$ and $|V\rangle$ denote horizontal and vertical photon polarization states. A photon pair in the $|\Phi^+\rangle$ state, when measured by Alice and Bob in the *same* polarization basis, always yields the same binary outcome (both H or both V , with equal probability).

A.2 Measurement Bases

Two mutually unbiased bases are used:

- **Z-basis (Rectilinear):** $\{|H\rangle, |V\rangle\}$, corresponding to bit values $\{0, 1\}$.
- **X-basis (Diagonal):** $\{|+\rangle = \frac{|H\rangle+|V\rangle}{\sqrt{2}}, |-\rangle = \frac{|H\rangle-|V\rangle}{\sqrt{2}}\}$, also mapped to $\{0, 1\}$.

For a $|\Phi^+\rangle$ pair, same-basis measurements are perfectly correlated. Different-basis measurements yield completely uncorrelated results with probability $\frac{1}{2}$ each — these trials carry no key information and

are discarded during sifting.

A.3 Security of BBM92

The security of BBM92 follows from the violation of Bell’s inequalities by entangled quantum states. An eavesdropper Eve who intercepts a photon must perform a measurement, collapsing the entangled state and destroying the Alice-Bob correlations. This introduces detectable errors into the sifted key. The threshold QBER of 11% corresponds to the maximum error rate consistent with no information leakage to a classical eavesdropper performing individual attacks.

A.4 Spontaneous Parametric Down-Conversion

The entangled photon source models SPDC in a Type-II nonlinear crystal (BBO). A pump photon at frequency ω_p and wavevector $\mathbf{k}_p$ is converted into a signal-idler pair:

$$\omega_p = \omega_s + \omega_i \quad (\text{energy conservation}) \quad (15)$$

$$\mathbf{k}_p = \mathbf{k}_s + \mathbf{k}_i \quad (\text{momentum conservation / phase matching}) \quad (16)$$

Type-II phase matching produces orthogonally polarized signal and idler photons. With appropriate compensation optics, the output state is the maximally entangled $|\Phi^+\rangle$ Bell state. The number of pairs emitted per pulse follows a Poisson distribution with mean $\langle n \rangle = \eta P_{\text{pump}} / \hbar \omega_p$, where η is the nonlinear conversion efficiency.

B Appendix B — Full Implementation Reference

B.1 Repository Structure

quantum-qkd/


```
|-- backend/
|   |-- main.py          # FastAPI server, simulation engines, custom classes
|   |-- README.md        # Backend API reference
|   +-- requirements.txt
|-- frontend/
|   |-- src/
|   |   |-- App.jsx      # React application (canvas, GIS map, inspector, playback)
|   |   |-- App.css      # Design system: dark mode, animations, layout
|   |   +-- main.jsx
|   +-- package.json
|-- DES/
|   |-- des.py           # Store DES tutorial script
|   +-- des.ipynb        # Interactive Jupyter notebook
|-- bbm92/
|   |-- bbm92.py         # Standalone BBM92 simulation script
|   +-- bbm92.log        # Sample execution logs
|-- entanglement/
|   +-- entanglement.py # Barrett-Kok protocol simulation
|-- hardware-module/
|   |-- hardware.py      # Quantum memory & detector component prototypes
|   +-- pingpong.py      # Classical protocol prototype
|-- resource-management/
|   +-- resource.py      # MemoryArray and ResourceManager integration
+-- reports/
    |-- week1.tex ... week5.tex # Weekly progress reports
    |-- final_report.tex        # This document
    +-- sources/0_frontpage.tex
```

B.2 Backend API

Table 7: REST API endpoints exposed by the FastAPI backend.

Method	Endpoint	Description
POST	/api/simulate	Accepts topology JSON; returns trial data, sifted keys, QBER, logs.
GET/HEAD	/api/health	Health check; returns simulation mode information.
GET/HEAD	/	Root status endpoint.

B.3 Simulation Request Schema

```

1 {
2   "numTrials": 100,
3   "selectedPair": {
4     "alice": "<node_id_of_alice>",
5     "bob":   "<node_id_of_bob>"
6   },
7   "nodes": [
8     {
9       "id": "source_node", "name": "Source", "type": "source",
10      "components": [
11        {
12          "id": "spdc1", "name": "SPDC Source", "type": "SPDCSource",
13          "mean_photon_num": 10.0, "frequency": 100
14        }
15      ]
16    },
17    {
18      "id": "alice_node", "name": "Alice", "type": "endpoint",
19      "components": [
20        {

```

```
21     "id": "det1", "name": "Detector A", "type": "Detector",
22     "efficiency": 0.95, "dark_count": 1e-6
23   }
24 ]
25 }
26 ],
27 "channels": [
28   {
29     "id": "qch1", "name": "Source to Alice", "type": "quantum",
30     "src": "source_node", "dst": "alice_node",
31     "distance": 1000.0, "attenuation": 0.0002,
32     "fidelity": 0.95, "delay": 5e-6
33   }
34 ]
35 }
```

Listing 9: Full simulation request JSON schema

B.4 Simulation Response Schema

The response object contains:

- `trials`: Array of per-trial objects with `trial`, `alice_basis`, `bob_basis`, `alice_result`, `bob_result`, `lossA`, `lossB`, `bsm` (if BSM node present), and `hops`.
- `siftedKeyAlice`, `siftedKeyBob`: Binary string representations of the sifted keys.
- `qber`: QBER as a percentage.
- `logs`: Array of human-readable simulation log strings.

- `alicePath`, `bobPath`: Lists of node IDs representing the discovered paths from source to Alice and Bob.

C Appendix C — BSM Correction Table

When an intermediate BSM node performs an entanglement swapping measurement, the Bob-side key bit must be corrected based on the BSM outcome before computing the sifted key.

Table 8: BSM outcome, resulting A-B entangled state, and required classical bit correction.

BSM Code	Projected Bell State (T_1, T_2)	A-B State	Bob's Correction
0	$ \Phi^+\rangle$	$\frac{1}{\sqrt{2}}(HH\rangle + VV\rangle)$	None
1	$ \Phi^-\rangle$	$\frac{1}{\sqrt{2}}(HH\rangle - VV\rangle)$	Bit flip: $b_B \leftarrow 1 - b_B$
2	$ \Psi^+\rangle$	$\frac{1}{\sqrt{2}}(HV\rangle + VH\rangle)$	None
3	$ \Psi^-\rangle$	$\frac{1}{\sqrt{2}}(HV\rangle - VH\rangle)$	Bit flip: $b_B \leftarrow 1 - b_B$

References

- [1] C. H. Bennett, G. Brassard, and N. D. Mermin, “Quantum cryptography based on Bell’s theorem,” *Physical Review Letters*, vol. 68, no. 5, p. 557, 1992.
- [2] SeQUeNCe Team, “SeQUeNCe: A Customizable Discrete-Event Simulator for Quantum Networks,” [Online]. Available: <https://sequence-toolbox.github.io/>, 2024.
- [3] C. Cicconetti, “NetSquid Tutorials,” [Online]. Available: <https://github.com/ccicconetti/netsquid>, 2024.
- [4] A. K. Ekert, “Quantum cryptography based on Bell’s inequalities,” *Physical Review Letters*, vol. 67, no. 6, p. 661, 1991.
- [5] P. G. Kwiat *et al.*, “New high-intensity source of polarization-entangled photon pairs,” *Physical Review Letters*, vol. 75, p. 4337, 1995.
- [6] M. Żukowski, A. Zeilinger, M. A. Horne, and A. K. Ekert, “‘Event-ready-detectors’ Bell experiment via entanglement swapping,” *Physical Review Letters*, vol. 71, p. 4287, 1993.
- [7] C. H. Bennett, G. Brassard, S. Popescu, B. Schumacher, J. A. Smolin, and W. K. Wootters, “Purification of noisy entanglement and faithful teleportation via noisy channels,” *Physical Review Letters*, vol. 76, no. 5, p. 722, 1996.
- [8] S. D. Barrett and P. Kok, “Efficient high-fidelity quantum computation using matter qubits and linear optics,” *Physical Review A*, vol. 71, p. 060310(R), 2005.
- [9] H.-K. Lo, M. Curty, and B. Qi, “Measurement-device-independent quantum key distribution,” *Physical Review Letters*, vol. 108, p. 130503, 2012.